Gestion d'un personnel universitaire

Exercice 3 — Héritage, polymorphisme et classes abstraites en C++

Compte rendu · Programmation orientée objet · Diagramme UML et implémentation C++17

1. Énoncé et plan de travail

Une université emploie trois catégories de personnel : des enseignants, des agents administratifs et des techniciens. Tous possèdent un matricule, un nom, un prénom et un salaire de base, mais chaque catégorie perçoit des indemnités ou des primes qui lui sont propres. L'objectif est de modéliser ce personnel par une hiérarchie de classes, puis de calculer et d'afficher le salaire de chacun de façon polymorphe.

Élément demandé	Traité en section
Identifier la classe mère et les classes dérivées	2
Réaliser le diagramme de classes UML et représenter l'héritage	3
Utiliser des méthodes virtuelles pour calculer le salaire	4 et 5
Implémenter les classes en C++	5
Afficher les informations de chaque employé	5 et 6
Créer une collection contenant plusieurs catégories d'employés	5 et 6
Notions mobilisées (héritage, polymorphisme, liaison dynamique…)	7

2. Analyse : identification des classes

2.1 Ce qui est commun, ce qui diffère

Les quatre informations communes (matricule, nom, prénom, salaire de base) sont factorisées dans une seule classe mère, `Employe`. Les éléments propres à chaque catégorie sont portés par les classes dérivées.

Classe	Rôle	Attributs
`Employe`	Classe mère abstraite	`matricule`, `nom`, `prenom`, `salaireBase` (communs)
`Enseignant`	Classe dérivée	`indemniteRecherche`, `indemniteEncadrement`, `heuresSup`, `tauxHeureSup`
`AgentAdministratif`	Classe dérivée	`primeRendement`, `primeResponsabilite`
`Technicien`	Classe dérivée	`primeTechnique`, `primeRisque`

2.2 Pourquoi l'héritage ?

Le test de la relation « est un » est satisfait : un enseignant est un employé, un agent administratif est un employé, un technicien est un employé. L'héritage public est donc le bon outil : il évite de dupliquer les attributs communs et permet de manipuler tous les employés à travers un même type, `Employe`.

2.3 Pourquoi une classe abstraite ?

Dans cette université, il n'existe pas d'employé « générique » : chacun appartient obligatoirement à l'une des trois catégories. De plus, le salaire ne peut être calculé que par la catégorie, puisque les indemnités diffèrent. `Employe` déclare donc `calculerSalaire()` comme méthode virtuelle pure : la classe devient abstraite, ne peut pas être instanciée et oblige chaque classe dérivée à fournir sa propre version.

3. Diagramme de classes UML

Le diagramme ci-dessous représente la hiérarchie : une classe mère abstraite et trois classes dérivées reliées par une relation d'héritage.

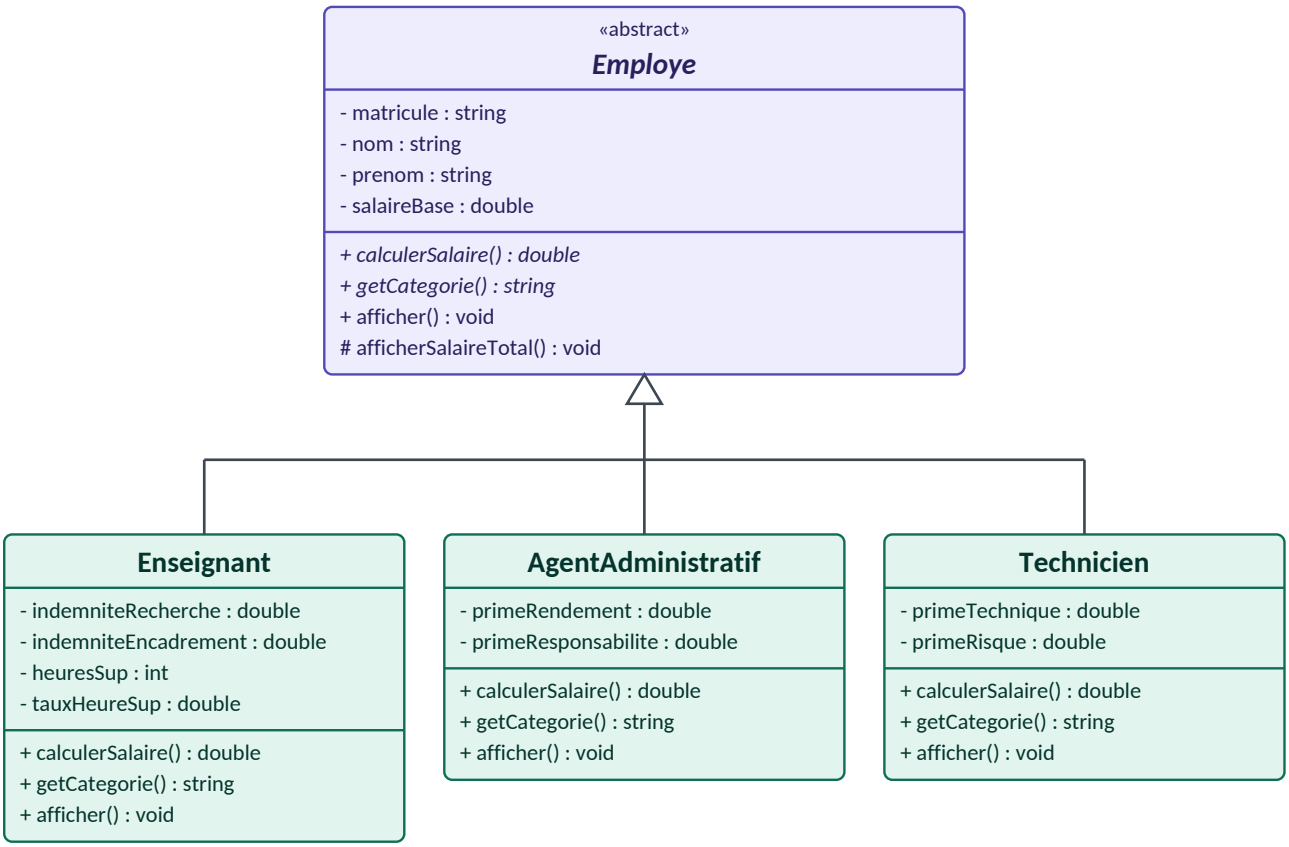


Figure 1 — Hiérarchie des classes du personnel universitaire

Lecture du diagramme

- Chaque classe est un rectangle à trois compartiments : nom, attributs, opérations.
- Les éléments abstraits sont en italique : la classe `Employe` et les opérations `calculerSalaire()` et `getCategorie()`.
- Visibilités : `+` public, `-` privé, `#` protégé.
- La flèche à tête triangulaire creuse est la relation d'héritage (généralisation). Elle part de la classe dérivée vers la classe mère : « `Enseignant` hérite de `Employe` ».
- Les trois classes dérivées redéfinissent les mêmes opérations : c'est cette redéfinition qui rend le polymorphisme possible. `afficherSalaireTotal()` est protégée : seules les classes dérivées l'utilisent.

4. Conception : calcul du salaire et polymorphisme

4.1 Formules de calcul

Catégorie	Salaire
`Enseignant`	salaire de base + indemnité de recherche + indemnité d'encadrement + (heures supplémentaires × taux horaire)
`AgentAdministratif`	salaire de base + prime de rendement + prime de responsabilité
`Technicien`	salaire de base + prime technique + prime de risque

4.2 Décisions de conception

Décision	Justification
Attributs privés, accesseurs publics	Encapsulation : les classes dérivées lisent le salaire de base par `getSalaireBase()` sans pouvoir le modifier.
`calculerSalaire()` et `getCategorie()` virtuelles pures	Contrat imposé à toute catégorie : impossible d'oublier de les redéfinir.
`afficher()` virtuelle avec comportement par défaut	La classe mère affiche l'identité et le salaire de base ; chaque classe dérivée appelle `Employe::afficher()` puis ajoute ses lignes. Aucune duplication.
Destructeur virtuel	Sans lui, détruire un objet dérivé via un `Employe*` n'appellerait pas le destructeur de la classe dérivée (comportement indéfini).
Utilitaires protégés	`montantValide`, `afficherLigne` et `afficherSalaireTotal` sont partagés par les classes dérivées et invisibles de l'extérieur.
Validation des données	Un montant négatif est refusé par une exception `std::invalid_argument` dès la construction.
Collection de `unique_ptr<Employe>`	Collection hétérogène sans `delete` manuel : la mémoire est libérée automatiquement.
Classes dérivées `final`	Aucune dérivation supplémentaire n'est prévue.

5. Implémentation C++

Le programme tient dans un seul fichier, `personnel_universite.cpp`, compilable avec g++ en C++17. Il est présenté en six parties ; les numéros de ligne renvoient au fichier fourni.

5.1 En-têtes et utilitaires

Les en-têtes fournissent les conteneurs (`<vector>`, `<map>`), les pointeurs intelligents (`<memory>`) et les algorithmes (`<algorithm>`, `<numeric>`). La fonction `longueurUtf8()` compte les caractères d'une chaîne plutôt que ses octets, afin que les colonnes restent alignées malgré les accents ; le bloc `#ifdef _WIN32` active l'UTF-8 dans la console Windows.

Fichier personnel_universite.cpp — lignes 1 à 37

```
/*
 * Exercice 3 — Gestion du personnel d'une université
 * ---------------------------------------------------
 * Notions : héritage, classe abstraite, méthodes virtuelles (pures),
 *           redéfinition (override), polymorphisme, liaison dynamique.
 *
 * Compilation : g++ -std=c++17 -Wall -Wextra -o personnel personnel_universite.cpp
 * Exécution   : ./personnel
 */

#include <algorithm>
#include <iomanip>
#include <iostream>
#include <map>
#include <memory>
#include <numeric>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

#ifdef _WIN32
#include <windows.h>   // pour afficher correctement les accents (UTF-8) dans la console
#endif

namespace {

const std::string DEVISE = "DA";          // à adapter : DH, €, ...
constexpr std::size_t LARGEUR_LIBELLE = 30;

// Longueur d'une chaîne UTF-8 en caractères (et non en octets) : alignement correct des accents.
std::size_t longueurUtf8(const std::string& s) {
    return static_cast<std::size_t>(std::count_if(
        s.begin(), s.end(), [](unsigned char c) { return (c & 0xC0) != 0x80; }));
}

}  // namespace
```

5.2 Classe mère abstraite Employe

Les attributs sont privés. La classe expose deux méthodes virtuelles pures (`= 0`), une méthode virtuelle ordinaire `afficher()` et un destructeur virtuel. Trois utilitaires protégés sont réservés aux classes dérivées.

Fichier personnel_universite.cpp — lignes 39 à 100

```cpp
// ============================================================================
//  CLASSE MÈRE ABSTRAITE : Employe
// ============================================================================
class Employe {
private:
    std::string matricule_;
    std::string nom_;
    std::string prenom_;
    double      salaireBase_;

protected:
    // Refuse les montants négatifs (utilisé par tous les constructeurs).
    static double montantValide(double valeur, const std::string& champ) {
        if (valeur < 0)
            throw std::invalid_argument("Valeur négative interdite pour : " + champ);
        return valeur;
    }

    // Affiche une ligne « libellé : montant » alignée.
    static void afficherLigne(const std::string& libelle, double montant) {
        const std::size_t n = longueurUtf8(libelle);
        std::cout << "    " << libelle
                  << std::string(n < LARGEUR_LIBELLE ? LARGEUR_LIBELLE - n : 0, ' ')
                  << ": " << std::fixed << std::setprecision(2) << std::setw(12) << montant
                  << ' ' << DEVISE << '\n';
    }

    // Affiche le salaire total : appel VIRTUEL à calculerSalaire().
    void afficherSalaireTotal() const {
        std::cout << "    " << std::string(LARGEUR_LIBELLE + 2 + 12 + 1 + DEVISE.size(), '-') << '\n';
        afficherLigne("SALAIRE TOTAL", calculerSalaire());
    }

public:
    Employe(std::string matricule, std::string nom, std::string prenom, double salaireBase)
        : matricule_(std::move(matricule)),
          nom_(std::move(nom)),
          prenom_(std::move(prenom)),
          salaireBase_(montantValide(salaireBase, "salaire de base")) {}

    // Destructeur VIRTUEL : indispensable pour détruire correctement un objet
    // dérivé via un pointeur de type Employe*.
    virtual ~Employe() = default;

    // ----- Accesseurs -----
    const std::string& getMatricule() const { return matricule_; }
    const std::string& getNom()       const { return nom_; }
    const std::string& getPrenom()    const { return prenom_; }
    double getSalaireBase()           const { return salaireBase_; }

    // ----- Méthodes virtuelles pures : chaque catégorie DOIT les redéfinir -----
    virtual double      calculerSalaire() const = 0;
    virtual std::string getCategorie()    const = 0;

    // ----- Méthode virtuelle « classique » : comportement par défaut, redéfinissable -----
    virtual void afficher() const {
        std::cout << "------------------------------------------------\n"
                  << "[" << getCategorie() << "] "
                  << prenom_ << " " << nom_ << "  (matricule " << matricule_ << ")\n";
        afficherLigne("Salaire de base", salaireBase_);
    }
};
```

5.3 Classe dérivée Enseignant

`Enseignant` redéfinit `calculerSalaire()`, `getCategorie()` et `afficher()`. Le mot-clé `override` demande au compilateur de vérifier la signature exacte ; `final` interdit toute dérivation supplémentaire. Le constructeur transmet l'identité et le salaire de base au constructeur de `Employe`.

Fichier personnel_universite.cpp — lignes 102 à 137

```
// ============================================================================
//  CLASSE DÉRIVÉE : Enseignant
// ============================================================================
class Enseignant final : public Employe {
private:
    double indemniteRecherche_;
    double indemniteEncadrement_;
    int    heuresSup_;
    double tauxHeureSup_;

public:
    Enseignant(std::string matricule, std::string nom, std::string prenom, double salaireBase,
               double indemniteRecherche, double indemniteEncadrement,
               int heuresSup, double tauxHeureSup)
        : Employe(std::move(matricule), std::move(nom), std::move(prenom), salaireBase),
          indemniteRecherche_(montantValide(indemniteRecherche, "indemnité de recherche")),
          indemniteEncadrement_(montantValide(indemniteEncadrement, "indemnité d'encadrement")),
          heuresSup_(static_cast<int>(montantValide(heuresSup, "heures supplémentaires"))),
          tauxHeureSup_(montantValide(tauxHeureSup, "taux horaire")) {}

    double calculerSalaire() const override {
        return getSalaireBase() + indemniteRecherche_ + indemniteEncadrement_
               + heuresSup_ * tauxHeureSup_;
    }

    std::string getCategorie() const override { return "Enseignant"; }

    void afficher() const override {
        Employe::afficher();   // réutilise l'affichage de la classe mère
        afficherLigne("Indemnité de recherche", indemniteRecherche_);
        afficherLigne("Indemnité d'encadrement", indemniteEncadrement_);
        afficherLigne("Heures sup. (" + std::to_string(heuresSup_) + " h)",
                      heuresSup_ * tauxHeureSup_);
        afficherSalaireTotal();
    }
};
```

5.4 Classe dérivée AgentAdministratif

Même schéma que `Enseignant`, avec une prime de rendement et une prime de responsabilité.

Fichier personnel_universite.cpp — lignes 139 à 166

```
// ============================================================================
//  CLASSE DÉRIVÉE : AgentAdministratif
// ============================================================================
class AgentAdministratif final : public Employe {
private:
    double primeRendement_;
    double primeResponsabilite_;

public:
    AgentAdministratif(std::string matricule, std::string nom, std::string prenom, double salaireBase,
                       double primeRendement, double primeResponsabilite)
        : Employe(std::move(matricule), std::move(nom), std::move(prenom), salaireBase),
          primeRendement_(montantValide(primeRendement, "prime de rendement")),
          primeResponsabilite_(montantValide(primeResponsabilite, "prime de responsabilité")) {}

    double calculerSalaire() const override {
        return getSalaireBase() + primeRendement_ + primeResponsabilite_;
    }

```

```
    std::string getCategorie() const override { return "Agent administratif"; }

    void afficher() const override {
        Employe::afficher();
        afficherLigne("Prime de rendement", primeRendement_);
        afficherLigne("Prime de responsabilité", primeResponsabilite_);
        afficherSalaireTotal();
    }
};
```

5.5 Classe dérivée Technicien

Même schéma, avec une prime technique et une prime de risque.

Fichier personnel_universite.cpp — lignes 168 à 195

```
// ============================================================================
//  CLASSE DÉRIVÉE : Technicien
// ============================================================================
class Technicien final : public Employe {
private:
    double primeTechnique_;
    double primeRisque_;

public:
    Technicien(std::string matricule, std::string nom, std::string prenom, double salaireBase,
               double primeTechnique, double primeRisque)
        : Employe(std::move(matricule), std::move(nom), std::move(prenom), salaireBase),
          primeTechnique_(montantValide(primeTechnique, "prime technique")),
          primeRisque_(montantValide(primeRisque, "prime de risque")) {}

    double calculerSalaire() const override {
        return getSalaireBase() + primeTechnique_ + primeRisque_;
    }

    std::string getCategorie() const override { return "Technicien"; }

    void afficher() const override {
        Employe::afficher();
        afficherLigne("Prime technique", primeTechnique_);
        afficherLigne("Prime de risque", primeRisque_);
        afficherSalaireTotal();
    }
};
```

5.6 Programme principal

`main()` crée la collection hétérogène (`std::vector<std::unique_ptr<Employe>>`) contenant six employés de trois catégories, puis l'exploite uniquement à travers le type `Employe` : affichage, masse salariale (`std::accumulate`), statistiques par catégorie (`std::map`), classement par salaire (`std::sort` sur des pointeurs non propriétaires) et test de validation.

Fichier personnel_universite.cpp — lignes 197 à 278

```
// ============================================================================
//  PROGRAMME PRINCIPAL
// ============================================================================
int main() {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif

    // Employe e("X", "Y", "Z", 1000);   // ERREUR de compilation : Employe est abstraite

    try {
        // Collection hétérogène : un seul conteneur, trois catégories d'employés.
        // unique_ptr => la mémoire est libérée automatiquement (via le destructeur virtuel).
        std::vector<std::unique_ptr<Employe>> personnel;
```

```
211
212         personnel.push_back(std::make_unique<Enseignant>(
213             "ENS-001", "Benali", "Karim", 60000, 12000, 8000, 10, 900));
214         personnel.push_back(std::make_unique<Enseignant>(
215             "ENS-002", "Haddad", "Amina", 72000, 15000, 10000, 6, 1100));
216         personnel.push_back(std::make_unique<AgentAdministratif>(
217             "ADM-001", "Mansouri", "Sofiane", 38000, 5000, 3000));
218         personnel.push_back(std::make_unique<AgentAdministratif>(
219             "ADM-002", "Kaci", "Lydia", 41000, 4500, 6000));
220         personnel.push_back(std::make_unique<Technicien>(
221             "TEC-001", "Ouali", "Yacine", 35000, 4000, 2500));
222         personnel.push_back(std::make_unique<Technicien>(
223             "TEC-002", "Bensaid", "Nadia", 37000, 4500, 3500));
224
225         // ---- 1. Affichage polymorphe (liaison dynamique) ----
226         std::cout << "==================== PERSONNEL ====================\n";
227         for (const auto& e : personnel)
228             e->afficher();          // la version appelée dépend du TYPE RÉEL de l'objet
229         std::cout << "---------------------------------------------------\n\n";
230
231         // ---- 2. Masse salariale totale ----
232         const double masse = std::accumulate(
233             personnel.begin(), personnel.end(), 0.0,
234             [](double acc, const auto& e) { return acc + e->calculerSalaire(); });
235         std::cout << "Masse salariale totale : " << std::fixed << std::setprecision(2)
236                   << masse << ' ' << DEVISE << "\n\n";
237
238         // ---- 3. Effectif et masse salariale par catégorie ----
239         std::map<std::string, std::pair<int, double>> parCategorie;
240         for (const auto& e : personnel) {
241             auto& stats = parCategorie[e->getCategorie()];
242             ++stats.first;
243             stats.second += e->calculerSalaire();
244         }
245         std::cout << "Répartition par catégorie :\n";
246         for (const auto& [categorie, stats] : parCategorie)
247             std::cout << "  " << std::left << std::setw(22) << categorie << std::right
248                       << stats.first << " employé(s)   "
249                       << std::setw(12) << stats.second << ' ' << DEVISE << '\n';
250
251         // ---- 4. Classement par salaire décroissant (pointeurs non propriétaires) ----
252         std::vector<const Employe*> classement;
253         for (const auto& e : personnel) classement.push_back(e.get());
254         std::sort(classement.begin(), classement.end(),
255                   [](const Employe* a, const Employe* b) {
256                       return a->calculerSalaire() > b->calculerSalaire();
257                   });
258         std::cout << "\nClassement par salaire décroissant :\n";
259         int rang = 1;
260         for (const Employe* e : classement)
261             std::cout << "  " << rang++ << ". " << e->getPrenom() << " " << e->getNom()
262                       << " (" << e->getCategorie() << ") - "
263                       << e->calculerSalaire() << ' ' << DEVISE << '\n';
264
265         // ---- 5. Contrôle des données : un salaire négatif est rejeté ----
266         std::cout << "\nTest de validation :\n";
267         try {
268             Technicien invalide("TEC-999", "Test", "Test", -500, 0, 0);
269         } catch (const std::invalid_argument& ex) {
270             std::cout << "  Création refusée -> " << ex.what() << '\n';
271         }
272     } catch (const std::exception& ex) {
273         std::cerr << "Erreur : " << ex.what() << '\n';
274         return 1;
275     }
276
277     return 0;
278 }
```

6. Exécution et résultats

Compilation et exécution :

```
$ g++ -std=c++17 -Wall -Wextra -pedantic -o personnel personnel_universite.cpp
$ ./personnel
```

Le programme compile sans aucun avertissement. Voici sa sortie complète :

```
=================== PERSONNEL ===================
-------------------------------------------------
[Enseignant] Karim Benali  (matricule ENS-001)
    Salaire de base               :     60000.00 DA
    Indemnité de recherche        :     12000.00 DA
    Indemnité d'encadrement       :      8000.00 DA
    Heures sup. (10 h)            :      9000.00 DA
    -----------------------------------------------
    SALAIRE TOTAL                 :     89000.00 DA
-------------------------------------------------
[Enseignant] Amina Haddad  (matricule ENS-002)
    Salaire de base               :     72000.00 DA
    Indemnité de recherche        :     15000.00 DA
    Indemnité d'encadrement       :     10000.00 DA
    Heures sup. (6 h)             :      6600.00 DA
    -----------------------------------------------
    SALAIRE TOTAL                 :    103600.00 DA
-------------------------------------------------
[Agent administratif] Sofiane Mansouri  (matricule ADM-001)
    Salaire de base               :     38000.00 DA
    Prime de rendement            :      5000.00 DA
    Prime de responsabilité       :      3000.00 DA
    -----------------------------------------------
    SALAIRE TOTAL                 :     46000.00 DA
-------------------------------------------------
[Agent administratif] Lydia Kaci  (matricule ADM-002)
    Salaire de base               :     41000.00 DA
    Prime de rendement            :      4500.00 DA
    Prime de responsabilité       :      6000.00 DA
    -----------------------------------------------
    SALAIRE TOTAL                 :     51500.00 DA
-------------------------------------------------
[Technicien] Yacine Ouali  (matricule TEC-001)
    Salaire de base               :     35000.00 DA
    Prime technique               :      4000.00 DA
    Prime de risque               :      2500.00 DA
    -----------------------------------------------
    SALAIRE TOTAL                 :     41500.00 DA
-------------------------------------------------
[Technicien] Nadia Bensaid  (matricule TEC-002)
    Salaire de base               :     37000.00 DA
    Prime technique               :      4500.00 DA
    Prime de risque               :      3500.00 DA
    -----------------------------------------------
    SALAIRE TOTAL                 :     45000.00 DA
-------------------------------------------------
Masse salariale totale : 376600.00 DA
Répartition par catégorie :
  Agent administratif   2 employé(s)       97500.00 DA
  Enseignant            2 employé(s)      192600.00 DA
  Technicien            2 employé(s)       86500.00 DA
Classement par salaire décroissant :
  1. Amina Haddad (Enseignant) - 103600.00 DA
  2. Karim Benali (Enseignant) - 89000.00 DA
  3. Lydia Kaci (Agent administratif) - 51500.00 DA
  4. Sofiane Mansouri (Agent administratif) - 46000.00 DA
  5. Nadia Bensaid (Technicien) - 45000.00 DA
  6. Yacine Ouali (Technicien) - 41500.00 DA
Test de validation :
  Création refusée -> Valeur négative interdite pour : salaire de base
```

Vérification par calcul manuel

Matricule	Nom	Catégorie	Calcul	Salaire
ENS-001	Benali Karim	Enseignant	60 000 + 12 000 + 8 000 + 10 × 900	89 000,00
ENS-002	Haddad Amina	Enseignant	72 000 + 15 000 + 10 000 + 6 × 1 100	103 600,00
ADM-001	Mansouri Sofiane	Agent administratif	38 000 + 5 000 + 3 000	46 000,00
ADM-002	Kaci Lydia	Agent administratif	41 000 + 4 500 + 6 000	51 500,00
TEC-001	Ouali Yacine	Technicien	35 000 + 4 000 + 2 500	41 500,00
TEC-002	Bensaid Nadia	Technicien	37 000 + 4 500 + 3 500	45 000,00
	Masse salariale			**376 600,00**

Les valeurs affichées par le programme coïncident avec le calcul manuel, ce qui valide les trois versions de `calculerSalaire()`. Le dernier test de la sortie montre qu'un salaire négatif est refusé dès la construction de l'objet.

7. Notions mobilisées

Notion	Définition	Dans le programme
Héritage	Une classe dérivée reprend les attributs et méthodes d'une classe mère.	`class Enseignant final : public Employe`
Classe abstraite	Classe non instanciable car elle contient au moins une méthode virtuelle pure.	`Employe` (`calculerSalaire() = 0`)
Méthode virtuelle	Méthode dont la version appelée dépend du type réel de l'objet.	`virtual void afficher() const`
Redéfinition	Une classe dérivée fournit sa propre version d'une méthode virtuelle.	`double calculerSalaire() const override`
Polymorphisme	Un même appel produit des comportements différents selon l'objet.	`e->afficher()` sur un `Employe*`, dans une boucle
Liaison dynamique	L'appel est résolu à l'exécution, grâce à la table virtuelle de l'objet.	`getCategorie()` dans `Employe::afficher()` ; `calculerSalaire()` dans `std::accumulate`

7.1 Que se passe-t-il à l'appel e->afficher() ?

1. Le type statique de `e` est `Employe` : le compilateur sait seulement que l'objet possède une méthode `afficher()`.
2. Comme `afficher()` est virtuelle, le compilateur ne fixe pas l'adresse de la fonction : il génère un accès à la table virtuelle (vtable) de l'objet.
3. À l'exécution, chaque objet pointe vers la vtable de sa classe réelle (`Enseignant`, `AgentAdministratif` ou `Technicien`) : la bonne version est trouvée et appelée.

4. À l'intérieur, `Employe::afficher()` appelle `getCategorie()` et `afficherSalaireTotal()` appelle `calculerSalaire()` : ces appels sont eux aussi dynamiques.

> **À retenir.** Sans le mot-clé `virtual`, l'appel serait résolu à la compilation (liaison statique) : tous les employés seraient affichés avec la version de `Employe`, et leurs indemnités spécifiques ne seraient jamais affichées.

8. Conclusion et extensions

La hiérarchie répond à l'ensemble de l'énoncé : une classe mère abstraite factorise l'identité et le salaire de base, chaque catégorie calcule son salaire par redéfinition d'une méthode virtuelle, et une unique collection permet de traiter tout le personnel sans jamais tester le type des objets. Le principal intérêt de cette conception est son extensibilité :

- ajouter une catégorie (vacataire, chercheur…) revient à écrire une nouvelle classe dérivée ; la boucle d'affichage, la masse salariale et le classement n'ont pas à être modifiés (principe ouvert/fermé) ;
- le personnel pourrait être chargé depuis un fichier CSV grâce à une fabrique qui crée l'objet correspondant à la catégorie lue ;
- la surcharge de `operator<<` permettrait d'écrire `std::cout << *e` pour afficher un employé.